

ML307C

LwM2M开发指导手册

版本：V1.0.0

发布日期：2025/3/7

服务与支持

如果您有任何关于模组产品及产品手册的评论、疑问、想法，或者任何无法从本手册中找到答案的疑问，请通过以下方式联系我们。

OneMO官网： onemo10086.com

邮箱： SmartModule@cmiot.chinamobile.com

客户服务热线： 400-110-0866



中国移动
China Mobile

文档声明

注意

本手册描述的产品及其附件特性和功能，取决于当地网络设计或网络性能，同时也取决于用户预先安装的各种软件。由于当地网络运营商、ISP，或当地网络设置等原因，可能也会造成本手册中描述的全部或部分产品及其附件特性和功能未包含在您的购买或使用范围之内。

责任限制

除非合同另有约定，中移物联网有限公司对本文档内容不做任何明示或暗示的声明或保证，并且不对特定目的适销性及适用性或者任何间接的、特殊的或连带的损失承担任何责任。

在适用法律允许的范围内，在任何情况下，中移物联网有限公司均不对用户因使用本手册内容和本手册中描述的产品而引起的任何特殊的、间接的、附带的或后果性的损坏、利润损失、数据丢失、声誉和预期的节省而负责。

因使用本手册中所述的产品而引起的中移物联网有限公司对用户的最大赔偿（除在涉及人身伤害的情况中根据适用法律规定的损害赔偿外），不应超过用户为购买此产品而支付的金额。

由于产品版本升级或其他原因，本文档内容会不定期进行更新。除非另有约定，本文档仅作为使用指导，本文档中的所有陈述、信息和建议不构成任何明示或暗示的担保。公司保留随时修改本手册中任何信息的权利，无需进行提前通知且不承担任何责任。

商标声明



为中国移动注册商标。

本手册和本手册描述的产品中出现的其他商标、产品名称、服务名称和公司名称，均为其各自所有者的财产。

进出口法规

出口、转口或进口本手册中描述的产品（包括但不限于产品软件和技术数据），用户应遵守相关进出口法律和法规。

隐私保护

关于我们如何保护用户的个人信息等隐私情况，请查看相关隐私政策。

操作系统更新声明

操作系统仅支持官方升级；如用户自己刷非官方系统，导致安全风险和损失由用户负责。

固件包完整性风险声明

固件仅支持官方升级；如用户自己刷非官方固件，导致安全风险和损失由用户负责。

版权所有©中移物联网有限公司。保留一切权利。

本手册中描述的产品，可能包含中移物联网有限公司及其存在的许可人享有版权的软件，除非获得相关权利人的许可，否则，非经本公司书面同意，任何单位和个人不得擅自摘抄、复制本手册内容的部分或全部，并以任何形式传播。



中国移动
China Mobile

关于文档

修订记录

版本	描述
V1.0.0	初版



中国移动
China Mobile

目录

服务与支持.....	ii
文档声明.....	iii
关于文档.....	v
表格索引.....	7
图示索引.....	
1. 概述.....	8
1.1. 适用型号.....	8
2. 功能说明.....	9
2.1. 创建和删除Lwm2m设备.....	10
2.2. 添加和删除OMA Object.....	11
2.3. 资源列表设置.....	12
2.4. 登录、更新和退出登录.....	13
2.5. 上传数据.....	14
2.6. 响应平台操作.....	15
3. API说明.....	16
4. 应用指导.....	17
4.1. OneNET平台接入示例.....	18
4.2. 基于DTLS加密的OneNET平台接入示例.....	24
4.3. CTWing平台接入示例.....	25
5. 编程设计注意.....	29
6. 附录.....	30

China Mobile

表格索引

Table 1: 适用模组.....	8
Table 2: 缩略语.....	30



1. 概述

本文档介绍了OpenCPU SDK的LwM2M功能，以便于用户参照文档和SDK中的Demo程序完成应用程序编写。

1.1. 适用型号

Table 1. 适用模组

模组系列	模组子型号
ML307C	ML307C-DC-CN

2. 功能说明

本文档围绕LwM2M功能介绍了OpenCPU SDK下基于LwM2M协议连接LwM2M平台相关API接口和使用示例。用户可参照本文档和SDK中的示例程序进行应用程序编写，包括连接OneNET平台和CTWing平台。



中国移动
China Mobile

2.1. 创建和删除LwM2M设备

创建一个LwM2M设备实例

```
int32_t cm_lwm2m_create(cm_lwm2m_cfg_t *cfg, cm_lwm2m_handle_t* handle);
```

删除一个LwM2M设备实例

```
int32_t cm_lwm2m_delete(cm_lwm2m_handle_t handle);
```



中国移动
China Mobile

2.2. 添加和删除OMA Object

向指定设备示例添加一个对象（object）及其所属的实例（instance）

```
int32_t cm_lwm2m_add_obj(cm_lwm2m_handle_t handle, int32_t objid, uint8_t instances[], int32_t inscount, int32_t attcount, int32_t actcount);
```

删除指定对象（object）及其所属的实例（instance）和资源（resource）

```
int32_t cm_lwm2m_del_obj(cm_lwm2m_handle_t handle, int32_t objid);
```



中国移动
China Mobile

2.3. 资源列表设置

设置指定对象（object）的所需资源列表

```
int32_t cm_lwm2m_discover(cm_lwm2m_handle_t handle, int32_t objid, int32_t resources[], int32_t rescount);
```

2.4. 登录、更新和退出登录

向平台发起登录请求

```
int32_t cm_lwm2m_open(cm_lwm2m_handle_t handle, uint32_t timeout, uint32_t lifetime);
```

向平台发起更新请求

```
int32_t cm_lwm2m_update(cm_lwm2m_handle_t handle, uint32_t lifetime, bool update_object);
```

向平台发起退出登录请求

```
int32_t cm_lwm2m_close(cm_lwm2m_handle_t handle);
```

2.5. 上传数据

（单次或多次）组包待上传数据

```
int32_t cm_lwm2m_notify_packing(cm_lwm2m_handle_t handle, int32_t objid, int32_t insid, int32_t resid, int32_t type,  
char *value, int32_t len, int32_t content_type);
```

向平台上报已组包数据

```
int32_t cm_lwm2m_notify(cm_lwm2m_handle_t handle, int32_t mid);
```



中国移动
China Mobile

2.6. 响应平台操作

响应读操作请求

```
int32_t cm_lwm2m_read_rsp(cm_lwm2m_handle_t handle, int32_t mid, int32_t result, int32_t objid, int32_t insid,  
int32_t resid, int32_t type, char *value, int32_t len);
```

响应写操作请求

```
int32_t cm_lwm2m_write_rsp(cm_lwm2m_handle_t handle, int32_t mid, int32_t result);
```

响应命令操作请求

```
int32_t cm_lwm2m_execute_rsp(cm_lwm2m_handle_t handle, int32_t mid, int32_t result);
```

响应策略参数操作请求

```
int32_t cm_lwm2m_param_rsp(cm_lwm2m_handle_t handle, int32_t mid, int32_t result);
```

响应观测请求操作请求

```
int32_t cm_lwm2m_observe_rsp(cm_lwm2m_handle_t handle, int32_t mid, int32_t result);
```

3. API说明

SDK提供LwM2M协议的用户编程接口，具体接口定义和说明请查阅当前使用SDK版本中的【OpenCPU_API手册.chm】文档（SDK中docs路径下）。



中国移动
China Mobile

4. 应用指导

⚠ CAUTION: 本文档及SDK中提供的API及其相关说明、示例程序等与发布时所对应的平台最新版本保持一致，由平台侧迭代更新导致功能无法使用、示例代码不一致等问题，用户应知悉并接受。

4.1. OneNET平台接入示例

本章以通信模组接入OneNET平台为例描述常见应用示例，涉及OneNET平台接入流程、协议和网页操作说明及接入服务器等信息，请参考OneNET网站获取最新版本以避免由平台迭代更新引起的相关问题。

模组注册回调函数

```
static void __cm_lwm2m_event_cb(int32_t event, cm_lwm2m_cb_param_t param)
{
    cm_demo_printf("[%d %s]event:%d\n", param.handle, (char *)param.cb_param, event);
    switch(event)
    {
        case CM_LWM2M_EVENT_REG_SUCCESS:
        {
            cm_demo_printf("lwm2m register ok\n");
        }
        break;
        case CM_LWM2M_EVENT_REG_FAILED:
        {
            cm_demo_printf("lwm2m register fail\n");
        }
        break;
        case CM_LWM2M_EVENT_UNREG_DONE:
        {
            cm_demo_printf("lwm2m unregister done\n");
        }
        break;
        default:
        break;
    }
}
```

```
static void __cm_lwm2m_notify_cb(int32_t mid, cm_lwm2m_cb_param_t param)
{
    cm_demo_printf("[%d %s]mid:%d\n", param.handle, (char *)param.cb_param, mid);
}
```

```
static void __cm_lwm2m_read_cb(int32_t mid, int32_t objid, int32_t insid, int32_t resid, cm_lwm2m_cb_param_t param)
{
    cm_demo_printf("[%d %s]mid:%d objid:%d insid:%d resid:%d\n", param.handle, (char *)param.cb_param, mid, objid, insid, resid);
}
```

```
static void __cm_lwm2m_write_cb(int32_t mid, int32_t objid, int32_t insid, int32_t resid, int32_t type, int32_t is_over, char *data, int32_t len, cm_lwm2m_cb_param_t param)
{
    cm_demo_printf("[%d %s]mid:%d objid:%d insid:%d resid:%d type:%d is_over:%d len:%d data:%s\n", param.handle, (char *)param.cb_param, mid, objid, insid, resid, type, is_over, len, data);
}
```

```
int32_t ret = cm_lwm2m_write_rsp(param.handle, mid, CM_LWM2M_RESULT_204_CHANGED);
```

```
if (CM_LWM2M_SUCCESS != ret)
{
    cm_demo_printf("cm_lwm2m_write_rsp() fail, ret is %d\n", ret);
    return;
}
}
```

```
static void __cm_lwm2m_execute_cb(int32_t mid, int32_t objid, int32_t insid, int32_t resid, char *data, int32_t len, cm_lwm2m_cb_param_t param)
```

```

{
    cm_demo_printf("[%d %s]mid:%d objid:%d insid:%d resid:%d len:%d len:%s\n", param.handle, (char
*)param.cb_param, mid, objid, insid, resid, len, data);

    int32_t ret = cm_lwm2m_execute_rsp(param.handle, mid, CM_LWM2M_RESULT_204_CHANGED);

    if (CM_LWM2M_SUCCESS != ret)
    {
        cm_demo_printf("cm_lwm2m_execute_rsp() fail, ret is %d\n", ret);
        return;
    }
}

static void __cm_lwm2m_observe_cb(int32_t mid, int32_t observe, int32_t objid, int32_t insid, int32_t resid,
cm_lwm2m_cb_param_t param)
{
    cm_demo_printf("[%d %s]%d_%d_%d: %d\n", param.handle, (char *)param.cb_param, objid, insid, resid, observe);
}

static void __cm_lwm2m_discover_cb(int32_t mid, int32_t objid, cm_lwm2m_cb_param_t param)
{
    cm_demo_printf("[%d %s]discover mid:%d objid:%d \n", param.handle, (char *)param.cb_param, mid, objid);
}

static void __cm_lwm2m_params_cb(int32_t mid, int32_t objid, int32_t insid, int32_t resid, char *parameter,
int32_t len, cm_lwm2m_cb_param_t param)
{
    cm_demo_printf("[%d %s]discover mid:%d objid:%d insid:%d resid:%d len:%d parameter:%s\n", param.handle, (char
*)param.cb_param, mid, objid, insid, resid, resid, parameter);

    int32_t ret = cm_lwm2m_param_rsp(param.handle, mid, CM_LWM2M_RESULT_204_CHANGED);

    if (CM_LWM2M_SUCCESS != ret)
    {
        cm_demo_printf("cm_lwm2m_param_rsp() fail, ret is %d\n", ret);
        return;
    }
}
}

```

模组接入OneNET平台

▪ Demo代码

```

cm_lwm2m_cfg_t cfg = {0};
cfg.platform = CM_LWM2M_ONENET;
cfg.endpoint.pattern = 2;
cfg.endpoint.name = NULL;
cfg.host = "183.230.40.39";
cfg.flag = 3; //bootstrap服务器, OneNET不支持moniter功能
cfg.auth_code = NULL;
cfg.psk = NULL;
cfg.pskid = NULL;
cfg.auto_update = 1;
cfg.cb.onRead = __cm_lwm2m_read_cb;
cfg.cb.onWrite = __cm_lwm2m_write_cb;
cfg.cb.onExec = __cm_lwm2m_execute_cb;
cfg.cb.onObserve = __cm_lwm2m_observe_cb;
cfg.cb.onParams = __cm_lwm2m_params_cb;
cfg.cb.onEvent = __cm_lwm2m_event_cb;
cfg.cb.onNotify = __cm_lwm2m_notify_cb;
cfg.cb.onDiscover = __cm_lwm2m_discover_cb;

```

```

cfg.cb_param = "OneNET";

ret = cm_lwm2m_create(&cfg, &onenet_dev);

int32_t if (CM_LWM2M_SUCCESS != ret)
{
    cm_demo_printf("[OneNET] cm_lwm2m_create() fail, ret is %d\n", ret);
    return;
}

uint8_t instances_3303[1] = {1};
ret = cm_lwm2m_add_obj(onenet_dev, 3303, instances_3303, 1, 0, 0);

if (CM_LWM2M_SUCCESS != ret)
{
    cm_demo_printf("[OneNET] cm_lwm2m_add_obj() obj 3303 fail, ret is %d\n", ret);
    return;
}

uint8_t instances_3306[1] = {1};
ret = cm_lwm2m_add_obj(onenet_dev, 3306, instances_3306, 1, 0, 0);

if (CM_LWM2M_SUCCESS != ret)
{
    cm_demo_printf("[OneNET] cm_lwm2m_add_obj() obj 3306 fail, ret is %d\n", ret);
    return;
}

int32_t resoures_3303[2] = {5700, 5601};
ret = cm_lwm2m_discover(onenet_dev, 3303, resoures_3303, 2);

if (CM_LWM2M_SUCCESS != ret)
{
    cm_demo_printf("[OneNET] cm_lwm2m_discover() obj 3303 fail, ret is %d\n", ret);
    return;
}

int32_t resoures_3306[2] = {5852, 5853};
ret = cm_lwm2m_discover(onenet_dev, 3306, resoures_3306, 2);

if (CM_LWM2M_SUCCESS != ret)
{
    cm_demo_printf("[OneNET] cm_lwm2m_discover() obj 3306 fail, ret is %d\n", ret);
    return;
}

ret = cm_lwm2m_open(onenet_dev, 30, 3000);

if (CM_LWM2M_SUCCESS != ret)
{
    cm_demo_printf("[OneNET] cm_lwm2m_open() fail, ret is %d\n", ret);
    return;
}

cm_demo_printf("[OneNET] onenet_dev[%d] CHECKIN SUCC\n", onenet_dev);

```

■ 代码运行时调试串口信息示例



模组向OneNET平台发送消息

▪ Demo代码

```
int32_t ret = cm_lwm2m_notify_packing(onenet_dev, 3303, 0, 5700, 4, "20.8", strlen("20.8"), 4);
```

```
if (CM_LWM2M_SUCCESS != ret)
{
    cm_demo_printf("[OneNET] cm_lwm2m_notify_packing() fail, ret is %d\n", ret);
    return;
}
```

```
ret = cm_lwm2m_notify(onenet_dev, 1);
```

```
if (CM_LWM2M_SUCCESS != ret)
{
    cm_demo_printf("[OneNET] cm_lwm2m_notify() fail, ret is %d\n", ret);
    return;
}
```

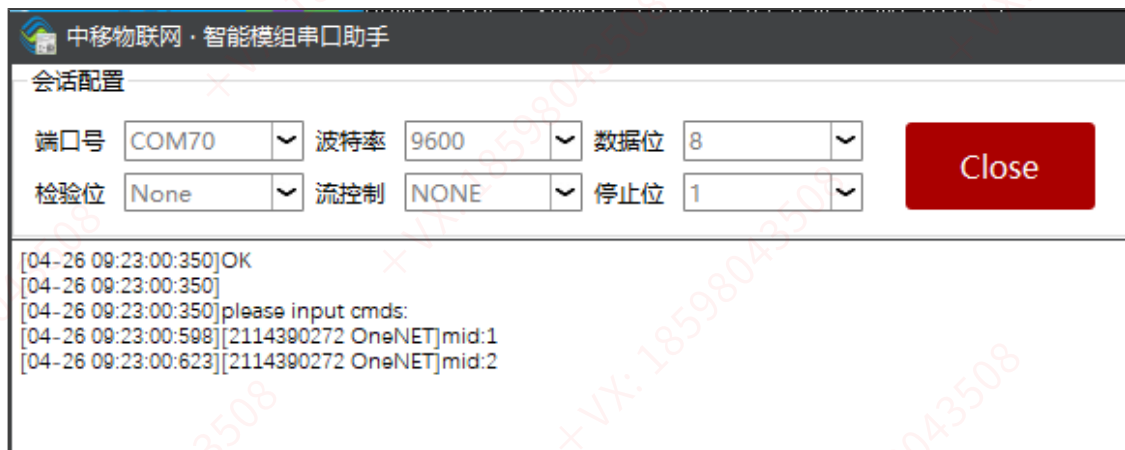
```
ret = cm_lwm2m_notify_packing(onenet_dev, 3306, 0, 5853, 1, "test", strlen("test"), 4);
```

```
if (CM_LWM2M_SUCCESS != ret)
{
    cm_demo_printf("[OneNET] cm_lwm2m_notify_packing() fail, ret is %d\n", ret);
    return;
}
```

```
ret = cm_lwm2m_notify(onenet_dev, 2);
```

```
if (CM_LWM2M_SUCCESS != ret)
{
    cm_demo_printf("[OneNET] cm_lwm2m_notify() fail, ret is %d\n", ret);
    return;
}
```

▪ 代码运行时调试串口信息示例



模组接收OneNET平台消息

- 代码运行时调试串口信息示例



模组退出OneNET平台

- Demo代码

```
ret = cm_lwm2m_del_obj(onenet_dev, 3303);

if (CM_LWM2M_SUCCESS != ret)
{
    cm_demo_printf("[OneNET] cm_lwm2m_del_obj() 3303 fail, ret is %d\n", ret);
    return;
}

ret = cm_lwm2m_del_obj(onenet_dev, 3306);

if (CM_LWM2M_SUCCESS != ret)
{
    cm_demo_printf("[OneNET] cm_lwm2m_del_obj() 3306 fail, ret is %d\n", ret);
    return;
}

ret = cm_lwm2m_close(onenet_dev);
```

```
if (CM_LWM2M_SUCCESS != ret)
{
    cm_demo_printf("[OneNET] cm_lwm2m_close() 3306 fail, ret is %d\n", ret);
    return;
}

ret = cm_lwm2m_delete(onenet_dev);

if (CM_LWM2M_SUCCESS != ret)
{
    cm_demo_printf("[OneNET] cm_lwm2m_delete() 3306 fail, ret is %d\n", ret);
    return;
}

onenet_dev = NULL;
```



4.2. 基于DTLS加密的OneNET平台接入示例

本节描述基于DTLS加密的OneNET应用示例。

模组基于DTLS加密创建OneNET实例

▪ Demo代码

```
cm_lwm2m_cfg_t cfg = {0};
cfg.platform = CM_LWM2M_ONENET;
cfg.endpoint.pattern = 2;
cfg.endpoint.name = NULL;
cfg.host = "183.230.40.39";
cfg.flag = 3; //bootstrap服务器, OneNET不支持moniter功能
cfg.auth_code = NULL; //与OneNET平台一致, 平台未设置时此值默认设置为NULL
cfg.psk = "wZnmpWzz0bzQupWF"; //与OneNET平台一致
cfg.pskid = NULL; //OneNET平台可将pskid默认设置为NULL
cfg.auto_update = 1;
cfg.cb.onRead = __cm_lwm2m_read_cb;
cfg.cb.onWrite = __cm_lwm2m_write_cb;
cfg.cb.onExec = __cm_lwm2m_execute_cb;
cfg.cb.onObserve = __cm_lwm2m_observe_cb;
cfg.cb.onParams = __cm_lwm2m_params_cb;
cfg.cb.onEvent = __cm_lwm2m_event_cb;
cfg.cb.onNotify = __cm_lwm2m_notify_cb;
cfg.cb.onDiscover = __cm_lwm2m_discover_cb;
cfg.cb_param = "OneNET";

ret = cm_lwm2m_create(&cfg, &onenet_dev);

if (CM_LWM2M_SUCCESS != ret)
{
    cm_demo_printf("[OneNET] cm_lwm2m_create() fail, ret is %d\n", ret);
    return;
}
```

 **Note:** 基于DTLS加密的OneNET连接与普通OneNET连接差异仅在于int32_t

cm_lwm2m_create(cm_lwm2m_cfg_t *cfg, cm_lwm2m_handle_t* handle);传入的cfg入参不同。其中cfg.auth_code须与OneNET平台一致, 平台未设置时此值默认设置为NULL, cfg.psk须与OneNET平台一致, cfg.pskid可默认设置为NULL。

4.3. CTWing平台接入示例

本节以通信模组接入CTWing平台为例描述常见的CTWing应用示例。

模组注册回调函数

```
static void __cm_lwm2m_event_cb(int32_t event, cm_lwm2m_cb_param_t param)
{
    cm_demo_printf("[%d %s]event:%d\n", param.handle, (char *)param.cb_param, event);
    switch(event)
    {
        case CM_LWM2M_EVENT_REG_SUCCESS:
        {
            cm_demo_printf("lwm2m register ok\n");
        }
        break;
        case CM_LWM2M_EVENT_REG_FAILED:
        {
            cm_demo_printf("lwm2m register fail\n");
        }
        break;
        case CM_LWM2M_EVENT_UNREG_DONE:
        {
            cm_demo_printf("lwm2m unregister done\n");
        }
        break;
        default:
        break;
    }
}

static void __cm_lwm2m_notify_cb(int32_t mid, cm_lwm2m_cb_param_t param)
{
    cm_demo_printf("[%d %s]mid:%d\n", param.handle, (char *)param.cb_param, mid);
}

static void __cm_lwm2m_read_cb(int32_t mid, int32_t objid, int32_t insid, int32_t resid, cm_lwm2m_cb_param_t param)
{
    cm_demo_printf("[%d %s]mid:%d objid:%d insid:%d resid:%d\n", param.handle, (char *)param.cb_param, mid, objid, insid, resid);
}

static void __cm_lwm2m_write_cb(int32_t mid, int32_t objid, int32_t insid, int32_t resid, int32_t type, int32_t is_over, char *data, int32_t len, cm_lwm2m_cb_param_t param)
{
    cm_demo_printf("[%d %s]mid:%d objid:%d insid:%d resid:%d type:%d is_over:%d len:%d data:%s\n", param.handle, (char *)param.cb_param, mid, objid, insid, resid, type, is_over, len, data);

    int32_t ret = cm_lwm2m_write_rsp(param.handle, mid, CM_LWM2M_RESULT_204_CHANGED);

    if (CM_LWM2M_SUCCESS != ret)
    {
        cm_demo_printf("cm_lwm2m_write_rsp() fail, ret is %d\n", ret);
        return;
    }
}

static void __cm_lwm2m_execute_cb(int32_t mid, int32_t objid, int32_t insid, int32_t resid, char *data, int32_t len, cm_lwm2m_cb_param_t param)
{

```

```

    cm_demo_printf("[%d %s]mid:%d objid:%d insid:%d resid:%d len:%d len:%s\n", param.handle, (char
*)param.cb_param, mid, objid, insid, resid, len, data);

    int32_t ret = cm_lwm2m_execute_rsp(param.handle, mid, CM_LWM2M_RESULT_204_CHANGED);

    if (CM_LWM2M_SUCCESS != ret)
    {
        cm_demo_printf("cm_lwm2m_execute_rsp() fail, ret is %d\n", ret);
        return;
    }
}

static void __cm_lwm2m_observe_cb(int32_t mid, int32_t observe, int32_t objid, int32_t insid, int32_t resid,
cm_lwm2m_cb_param_t param)
{
    cm_demo_printf("[%d %s]%d_%d_%d: %d\n", param.handle, (char *)param.cb_param, objid, insid, resid, observe);
}

static void __cm_lwm2m_discover_cb(int32_t mid, int32_t objid, cm_lwm2m_cb_param_t param)
{
    cm_demo_printf("[%d %s]discover mid:%d objid:%d \n", param.handle, (char *)param.cb_param, mid, objid);
}

static void __cm_lwm2m_params_cb(int32_t mid, int32_t objid, int32_t insid, int32_t resid, char *parameter,
int32_t len, cm_lwm2m_cb_param_t param)
{
    cm_demo_printf("[%d %s]discover mid:%d objid:%d insid:%d resid:%d len:%d parameter:%s\n", param.handle, (char
*)param.cb_param, mid, objid, insid, resid, resid, parameter);

    int32_t ret = cm_lwm2m_param_rsp(param.handle, mid, CM_LWM2M_RESULT_204_CHANGED);

    if (CM_LWM2M_SUCCESS != ret)
    {
        cm_demo_printf("cm_lwm2m_param_rsp() fail, ret is %d\n", ret);
        return;
    }
}

```

模组接入CTWing 平台

▪ Demo代码

```

cm_lwm2m_cfg_t cfg = {0};
cfg.platform = CM_LWM2M_CTWING;
cfg.endpoint.pattern = 1;
cfg.endpoint.name = NULL;
cfg.host = "221.229.214.202";
cfg.flag = 0;
cfg.auth_code = NULL;
cfg.psk = NULL;
cfg.pskid = NULL;
cfg.auto_update = 1;
cfg.cb.onRead = __cm_lwm2m_read_cb;
cfg.cb.onWrite = __cm_lwm2m_write_cb;
cfg.cb.onExec = __cm_lwm2m_execute_cb;
cfg.cb.onObserve = __cm_lwm2m_observe_cb;
cfg.cb.onParams = __cm_lwm2m_params_cb;
cfg.cb.onEvent = __cm_lwm2m_event_cb;
cfg.cb.onNotify = __cm_lwm2m_notify_cb;
cfg.cb.onDiscover = __cm_lwm2m_discover_cb;
cfg.cb_param = "CTWing";

```

```
ret = cm_lwm2m_create(&cfg, &onenet_dev);
```

```
if (CM_LWM2M_SUCCESS != ret)
{
    cm_demo_printf("[CTWing] cm_lwm2m_create() fail, ret is %d\n", ret);
    return;
}
```

```
uint8_t instances_19[1] = {1};
ret = cm_lwm2m_add_obj(onenet_dev, 19, instances_19, 1, 0, 0);
```

```
if (CM_LWM2M_SUCCESS != ret)
{
    cm_demo_printf("[CTWing] cm_lwm2m_add_obj() obj 19 fail, ret is %d\n", ret);
    return;
}
```

```
int32_t resoures_19[1] = {0};
ret = cm_lwm2m_discover(onenet_dev, 19, resoures_19, 1);
```

```
if (CM_LWM2M_SUCCESS != ret)
{
    cm_demo_printf("[CTWing] cm_lwm2m_discover() obj 19 fail, ret is %d\n", ret);
    return;
}
```

```
ret = cm_lwm2m_open(onenet_dev, 30, 3000);
```

```
if (CM_LWM2M_SUCCESS != ret)
{
    cm_demo_printf("[CTWing] cm_lwm2m_open() fail, ret is %d\n", ret);
    return;
}
```

```
cm_demo_printf("[CTWing] onenet_dev[%d] CHECKIN SUCC\n", onenet_dev);
```

代码运行时调试串口信息示例

```
[2024-04-26_15:16:33:792][CTWing] onenet_dev[2115225536] CHECKIN SUCC
[2024-04-26_15:16:33:792]OK

[2024-04-26_15:16:33:792]please input cmds:
[2024-04-26_15:16:34:153][2115225536 CTWing]event:6
[2024-04-26_15:16:34:153]lwm2m register ok
[2024-04-26_15:16:34:153][2115225536 CTWing]19_0_0: 1
```

模组向CTWing平台发送消息

Demo代码

```
int32_t ret = cm_lwm2m_notify_packing(onenet_dev, 19, 0, 0, 1, "test", strlen("test"), 1);
```

```
if (CM_LWM2M_SUCCESS != ret)
{
    cm_demo_printf("[CTWing] cm_lwm2m_notify_packing() fail, ret is %d\n", ret);
    return;
}
```

```
static int mid = 1;
ret = cm_lwm2m_notify(onenet_dev, mid++);
```

```
if (CM_LWM2M_SUCCESS != ret)
{
    cm_demo_printf("[CTWing] cm_lwm2m_notify() fail, ret is %d\n", ret);
    return;
}
```

- 代码运行时调试串口信息示例

```
[2024-04-26_15:17:09:435]please input cmds:
[2024-04-26_15:17:09:646][2115225536 CTWing]mid:1
[2024-04-26_15:17:39:022]OK
```

模组接收CTWing平台消息

- 代码运行时调试串口信息示例

```
[2024-04-26_15:17:39:265][2115225536 CTWing]mid:21606 objid:19 insid:1 resid:0
type:2 is_over:0 len:4 data:test
```

模组退出CTWing平台

- Demo代码

```
int32_t ret = cm_lwm2m_del_obj(onenet_dev, 19);
```

```
if (CM_LWM2M_SUCCESS != ret)
{
    cm_demo_printf("[CTWing] cm_lwm2m_del_obj() 19 fail, ret is %d\n", ret);
    return;
}
```

```
ret = cm_lwm2m_close(onenet_dev);
```

```
if (CM_LWM2M_SUCCESS != ret)
{
    cm_demo_printf("[CTWing] cm_lwm2m_close() fail, ret is %d\n", ret);
    return;
}
```

```
ret = cm_lwm2m_delete(onenet_dev);
```

```
if (CM_LWM2M_SUCCESS != ret)
{
    cm_demo_printf("[CTWing] cm_lwm2m_delete() fail, ret is %d\n", ret);
    return;
}
```

```
onenet_dev = NULL;
```

5. 编程设计注意

使用LwM2M访问OneNET平台或CTWing平台时应注意以下几点：

- 使用OneNET功能或CTWing功能前，应在平台上完成产品和设备创建；
- OneNET平台和CTWing平台仅支持一路实例，勿同时开启多路实例；
- OneNET功能和CTWing功能不可同时使用，功能切换前应注销设备与当前使用平台的连接且清理对应设备资源；
- LwM2M支持连接OneNET平台或CTWing平台相关操作，不支持LwM2M协议下的OneNET FOTA和CTWing FOTA相关操作，亦不支持与其他平台的连接和相关操作。

6. 附录

Table 2. 缩略语

缩写	英文全称	中文解释
LwM2M	Lightweight Machine to Machine	轻量级M2M协议
DTLS	Datagram Transport Layer Security	数据包传输层安全性协议